DEMOCRATIC AND POPULAR REPUBLIC OF ALGERIA
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

KASDI MERBAH UNIVERSITY – OUARGLA
FACULTY OF NEW INFORMATION AND COMMUNICATION TECHNOLOGIES
COMPUTER SCIENCE AND INFORMATION TECHNOLOGY DEPARTMENT



ADS1

CHAPTER 1

INTRODUCTION TO PROGRAMMING

Presented by:

Dr. Aicha KORICHI

Academic Year
2026 – 2027

Today's Plan

1. Introduction to the course
2. What is Computer Science?
3. What is a Computer?
4. Information Processing Cycle
5. Problem Solving and Algorithms
6. Algorithm Example and Activity
7. Representing Algorithms (Pseudocode & Flowchart)
8. From Algorithm to Program
9. Programming Languages and Why C?
10. First Look at a C Program
11. Summary and Questions



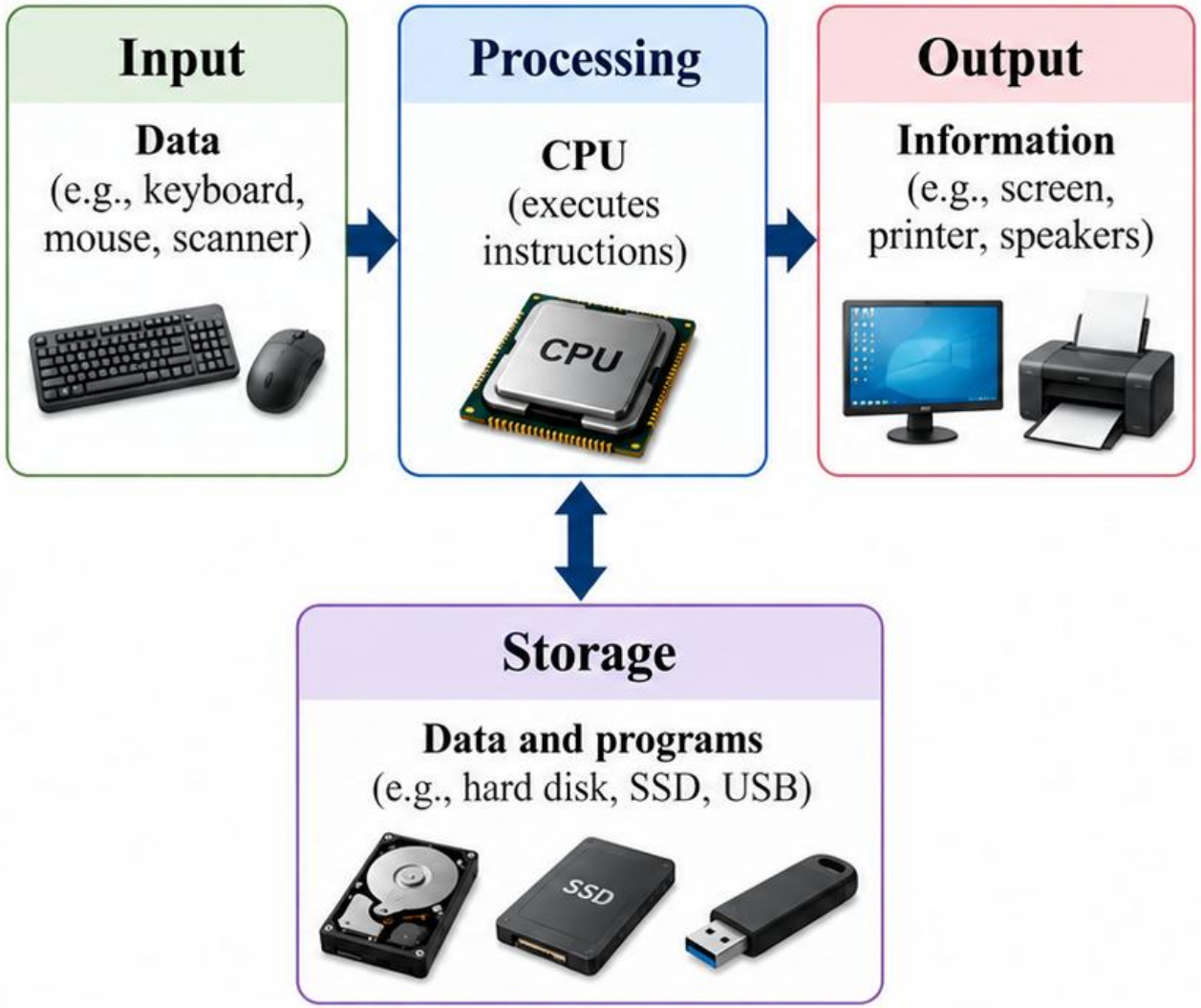


What is a Computer?

A **computer** is an electronic device that receives data, processes it according to a set of instructions, stores it, and produces useful information.

Main characteristics:

- **Speed:** performs millions of operations per second.
- **Accuracy:** produces reliable results.
- **Storage:** can store large amounts of data.
- **Versatility:** can execute different types of tasks (calculations, text, images, etc.).
- **Programmability:** follows a set of instructions (a program).



What is Computer Science?

Computer Science is the scientific discipline that studies the principles, methods, and tools for processing information using computers.

Main aspects of Computer Science:

- **Information representation:** how to represent data (numbers, text, images, etc.).
- **Algorithms:** methods to solve problems.
- **Programming:** writing instructions for a computer.
- **Data structures:** organizing and storing data efficiently.
- **Software development:** designing and building computer programs.
- **Applications:** using computers to solve real-world problems in various domains.

Computer Science combines:



Examples of applications:



Main Components of a Computer

A computer is built from several components that work together to perform the **input, processing, storage and output** of data.

1 Input devices

- Allow the user to enter data and instructions.
- Examples: keyboard, mouse, scanner, microphone…

2 Central Processing Unit (CPU)

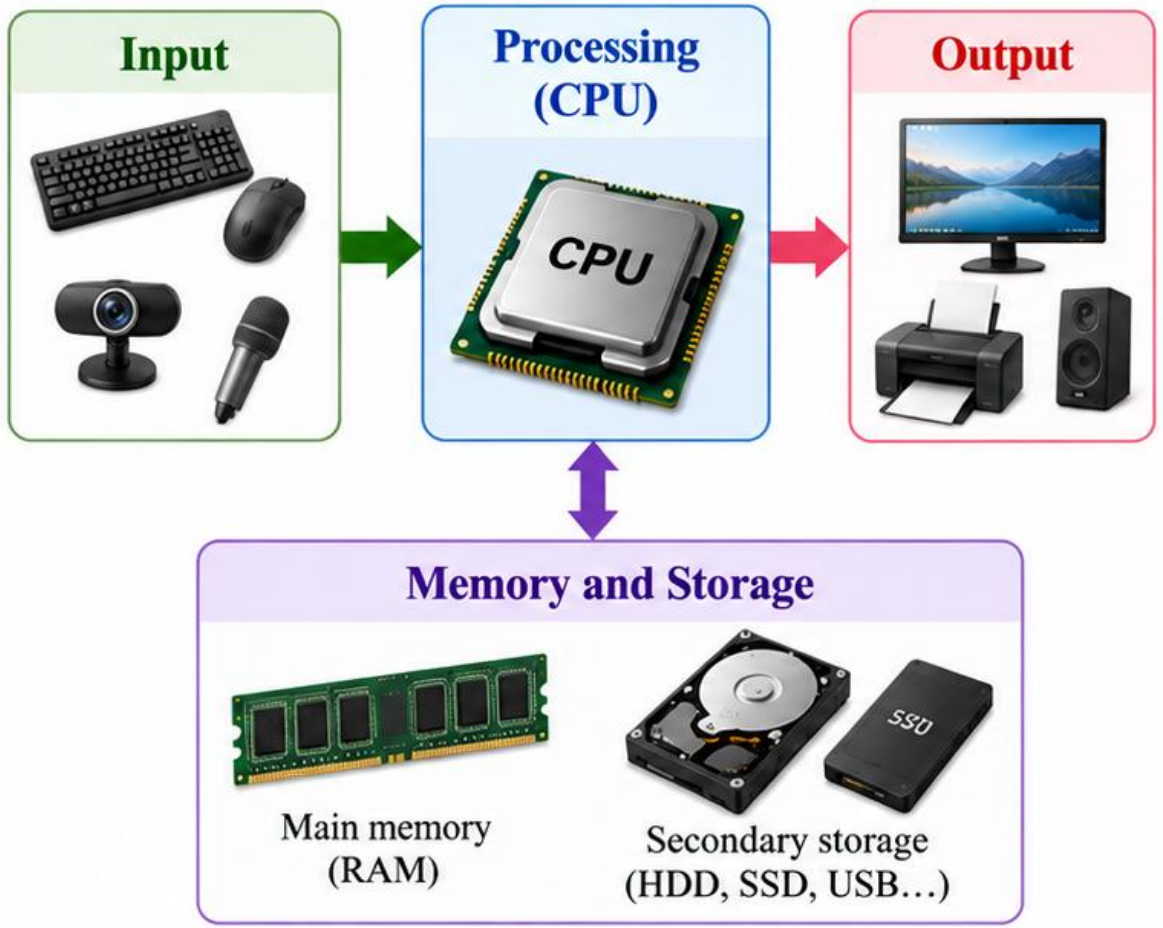
- Executes instructions and processes data.
- Made of two main parts: Control Unit and Arithmetic Logic Unit (ALU).

3 Memory and storage

- Temporary storage (main memory / RAM) for data and programs in use.
- Pernanent storage (secondary storage) for long-term storage.

4 Output devices

- Display the results produced by the computer.
- Examples: screen, printer, speakers, projector…



7. Information Processing Cycle

A **computer** processes data to transform it into useful information through a cycle of four main steps.

Input
(Data)

Raw data entered into the computer (e.g., keyboard, mouse, scanner).



Processing
(Instructions)

The CPU executes instructions on the data.



Output
(Information)

Useful results (e.g., text, images, reports).



Storage
(Data and programs)

Data and programs are stored for later use.



The cycle repeats as needed.

8. Problem Solving

A **problem** is a situation or a question that needs to be solved.

Examples of problems:

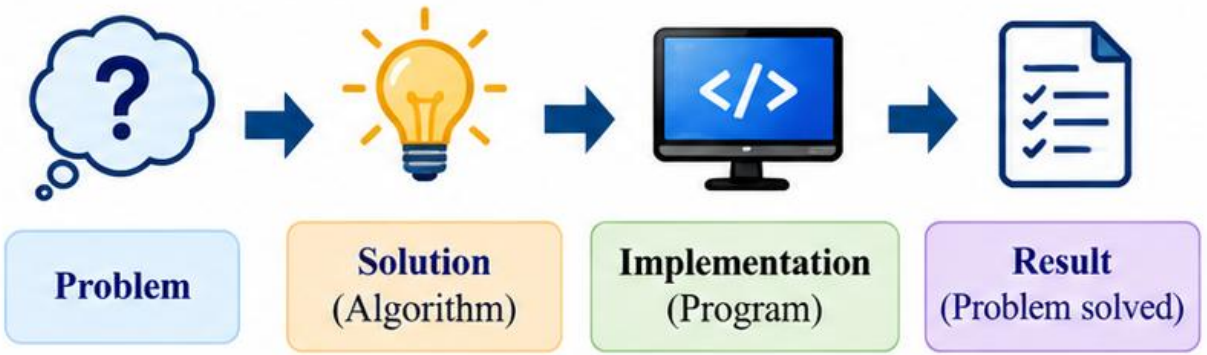
- Calculate the average of a set of numbers.
- Find the largest value in a list.
- Determine if a number is positive or negative.
- Search for a specific element.
- Manage student records.
- …

A problem can be characterized by:

- **Inputs:** the given data.
- **Outputs:** the expected results.
- **Conditions (constraints):** rules to be respected.
- A clear goal to be achieved.

Problem solving steps:

1. **Understand** the problem (identify inputs, outputs, and constraints).
2. **Design** a solution (algorithm).
3. **Implement** the solution (program).
4. **Test** and verify the solution.
5. **Refine** and improve if necessary.



9. What is an Algorithm?

An **algorithm** is a finite sequence of well-defined steps to solve a problem or perform a task.

Characteristics of an algorithm:

- Well-defined steps (clear and unambiguous).
- Finite (must terminate).
- Has input (zero or more).
- Produces output (one or more).
- Is effective (each step can be executed in a finite time).
- Is general (can be applied to similar problems).

Representation of an algorithm:

An algorithm can be represented using:

- Natural language (text).
- Pseudocode.
- Flowchart.

Example in pseudocode:

```
Algorithm Sum_Two_Numbers
Input: a, b
Output: s

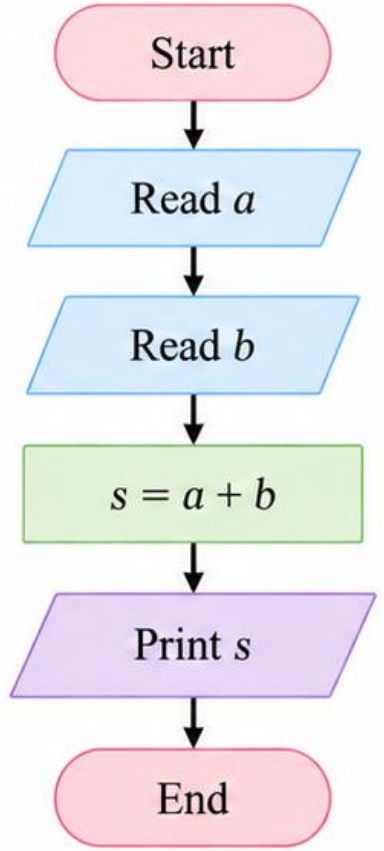
Begin
   Read a
   Read b
   s ← a + b
   Print s
End
```

Example: Find the sum of two numbers

Problem: Read two numbers and calculate their sum.

Algorithm (steps):

1. Start.
2. Read the first number (a).
3. Read the second number (b).
4. Compute the sum $s = a + b$.
5. Display the result.
6. End.



10. Representing Algorithms

An algorithm can be represented in different ways, such as **natural language**, **pseudocode**, and **flowcharts**.

1. Natural Language

An algorithm is described using simple and clear sentences in a human language.

Example: Compute the average of three numbers.

1. Read three numbers: a, b and c.
2. Compute the sum: $s = a + b + c$.
3. Compute the average: $avg = s / 3$.
4. Display the average.

2. Pseudocode

A pseudocode is a structured and informal representation of an algorithm, similar to a programming language.

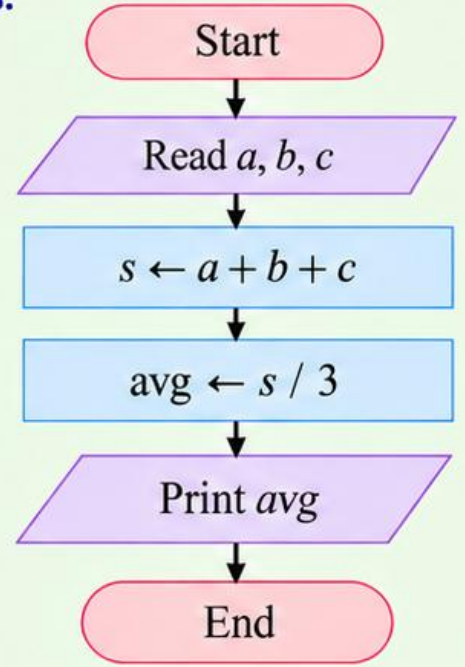
Example: Compute the average of three numbers.

```
Algorithm Average_Three_Numbers;
Var:  a : real;  b : real;  c : real;
      s : real;  avg : real;
Begin
      Read(a);
      Read(b);
      Read(c);
      s ← a + b + c;
      avg ← s / 3;
      Print(avg);
End;
```

3. Flowchart

A flowchart is a graphical representation of an algorithm using standard symbols.

Example: Compute the average of three numbers.



12. From an Algorithm to a Program

An algorithm is a **language-independent** description of a solution. To execute it on a computer, it must be **translated** into a programming language.

1. Development steps

1. Problem analysis
2. Design the algorithm (pseudocode or flowchart)
3. Translate into a programming language (e.g., C)
4. Compile and run the program
5. Test and validate the result

2. Example: Compute the average of three numbers

Pseudocode

```
Algorithm Average_Three_Numbers;
Var:  a : real; b : real; c : real;
      avg : real;
Begin
    Read(a);
    Read(b);
    Read(c);
    avg ← (a + b + c) / 3;
    Print(avg);
End;
```

C Program

```c
#include <stdio.h>
int main() {
    float a, b, c, avg;

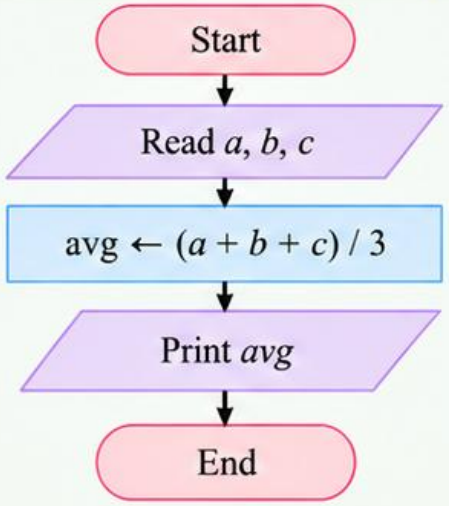
    printf("Enter three numbers: ");
    scanf("%f %f %f", &a, &b, &c);

    avg = (a + b + c) / 3;

    printf("Average = %.2f\n", avg);

    return 0;
}
```

3. Flowchart of the example



4. Key points

- The same algorithm can be implemented in **different programming languages**.
- Pseudocode and flowcharts help to **design** and **understand** the solution.
- A program is the **executable version** of the algorithm.
- **Testing** is important to verify that the program produces the correct result.

12. From an Algorithm to a Program

An algorithm is a **language-independent** description of a solution. To execute it on a computer, it must be **translated** into a programming language.

1. Development steps

1. Problem analysis
2. Design the algorithm (pseudocode or flowchart)
3. Translate into a programming language (e.g., C)
4. Compile and run the program
5. Test and validate the result

2. Example: Compute the average of three numbers

Pseudocode

```
Algorithm Average_Three_Numbers;
Var:  a : real; b : real; c : real;
      avg : real;
Begin
    Read(a);
    Read(b);
    Read(c);
    avg ← (a + b + c) / 3;
    Print(avg);
End;
```

C Program

```c
#include <stdio.h>
int main() {
    float a, b, c, avg;

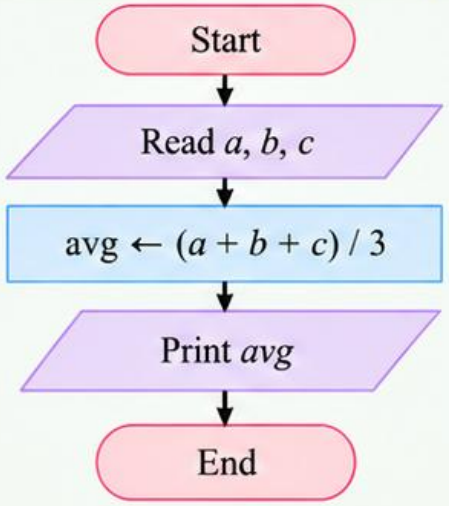
    printf("Enter three numbers: ");
    scanf("%f %f %f", &a, &b, &c);

    avg = (a + b + c) / 3;

    printf("Average = %.2f\n", avg);

    return 0;
}
```

3. Flowchart of the example



4. Key points

- The same algorithm can be implemented in **different programming languages**.
- Pseudocode and flowcharts help to **design** and **understand** the solution.
- A program is the **executable version** of the algorithm.
- **Testing** is important to verify that the program produces the correct result.

13. Example: Maximum of Three Numbers

1. Problem Statement

Write an algorithm to read three numbers and determine the largest one.

Input:

- Three numbers: a, b, c.

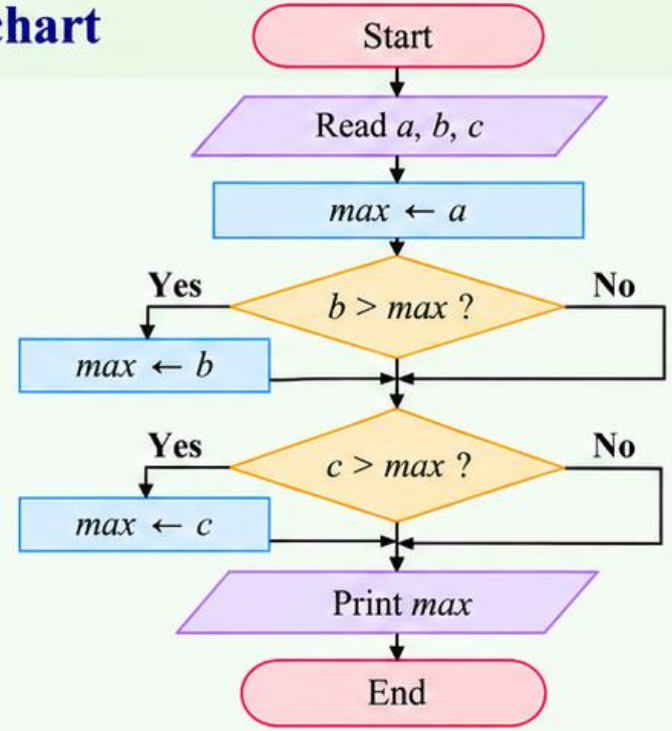
Output:

- The maximum value among a, b and c.

2. Algorithm in Pseudocode

```
Algorithm Maximum_Of_Three;
Var: a : real; b : real; c : real; max : real;
Begin
        Read(a);
        Read(b);
        Read(c);
        max ← a;
        If (b > max) then
                max ← b;
        End if;
        If (c > max) then
                max ← c;
        End if;
        Print(max);
End;
```

3. Flowchart



4. Trace Example

Input: $a = 12,\ b = 25,\ c = 7$

Step	*a*	*b*	*c*	*max*	Condition	Action
1	12	25	7	12	–	$max \leftarrow a$
2	12	25	7	25	$b > 12$ (true)	$max \leftarrow b$
3	12	25	7	25	$c > 25$ (false)	no change
4	12	25	7	25	–	print *max*

Output: The maximum is 25.

14. Algorithm vs Program

An **algorithm** and a **program** are related, but they are not the same. An algorithm describes the solution, while a program is **the implementation** of the algorithm in a programming language.

1. What is an Algorithm?

- An algorithm is a **finite** and **ordered** set of **well-defined steps** to solve a problem.
- It is a **language-independent** description of the solution.
- It can be represented using **natural language**, **pseudocode** or a **flowchart**.

Example: Compute the average of three numbers.

Pseudocode:

Algorithm Average_Three_Numbers;
Var: a : real; b : real; c : real; avg : real;
Begin
    Read(a);
    Read(b);
    Read(c);
    avg ← $(a + b + c) / 3$;
    Print(avg);
End;

2. What is a Program?

- A program is a set of instructions written in a **programming language** to be executed by a computer.
- It is the **implementation** of an algorithm.
- A program must respect the **syntax and rules** of the chosen language (e.g., C).
- It can be compiled and executed on a computer.

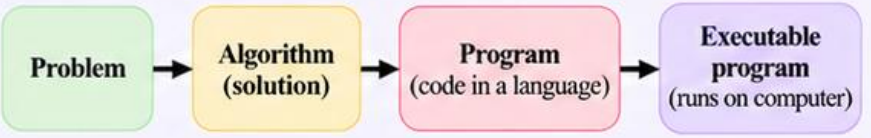
Example: The same algorithm written as a C program.

```c
#include <stdio.h>
int main() {
    float a, b, c, avg;
    printf("Enter three numbers: ");
    scanf("%f %f %f", &a, &b, &c);
    avg = (a + b + c)/ 3;
    printf("Average = %.2f\n", avg);
    return 0;
}
```

3. Key Differences

Aspect	Algorithm	Program
Nature	Solution / plan	Implementation
Form	Steps in natural language, pseudocode or flowchart	Instructions in a programming language
Language	Language-independent	Written in a specific programming language (e.g., C)
Execution	Not executable	Executable on a computer (after compilation)
Focus	Logic of the solution	Technical implementation
Example	Pseudocode / flowchart	C code

4. Relationship



15. What is a Programming Language?

A **programming language** is a formal language used to write programs that can be executed by a computer.

1. Definition

A programming language is a set of rules (**syntax** and **semantics**) that allows a programmer to write **instructions** to solve a problem and to control a computer.

2. Purpose

- To **express algorithms** in a form **understandable by a computer**.
- To **write, test** and modify programs.
- To **communicate** with the computer and use its resources.
- To develop **software applications**.

3. Main Elements of a Programming Language

- **Syntax:** defines the structure and form of valid instructions.
- **Semantics:** defines the meaning of the instructions.
- **Vocabulary:** set of keywords, operators, identifiers, etc.

4. Examples of Programming Languages



5. Example: The Same Instruction in Different Languages

All programming languages allow us to write instructions, but the syntax is different.

Language	Instruction: display a message
C	`printf("Hello World");`
C++	`std::cout << "Hello World";`
Python	`print("Hello World")`
Java	`System.out.println("Hello World");`
JavaScript	`console.log("Hello World");`

16. Types of Programming Languages

Programming languages can be classified into different types depending on their **level of abstraction** and how they are **translated** and **executed** by the computer.

1. Machine Language

- Written in binary code (0 and 1).
- Directly understood by the computer hardware (CPU).
- Difficult to write, read and maintain.
- Very fast execution.

Example:

```
10110000  01100001  10001010
10110100  00001111  11000001
```

2. Assembly Language

- Uses mnemonic codes (e.g., MOV, ADD).
- Closer to the hardware.
- Easier to understand than machine language, but still difficult compared to high-level languages.
- Translated into machine code using an assembler.

Example:

```
MOV  AX, 5
ADD  AX, BX
SUB  AX, 1
```

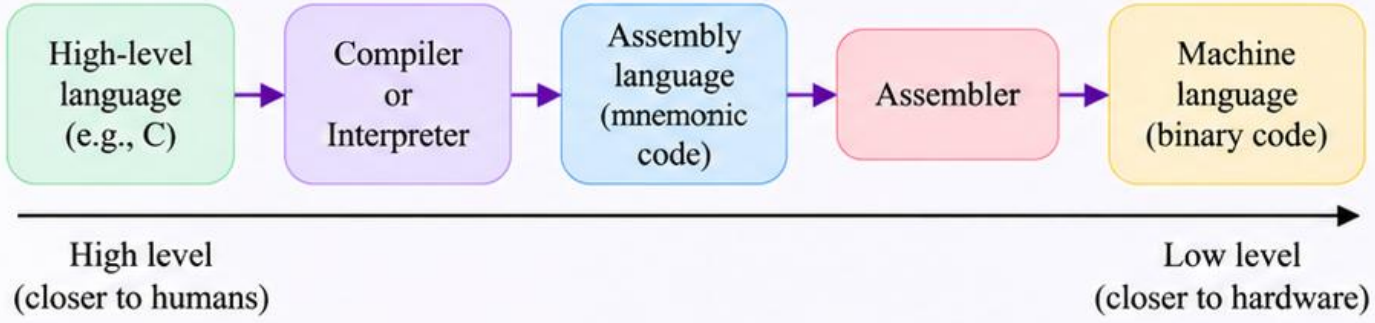
3. High-Level Languages

- Use a syntax close to natural language.
- Easier to write, read and maintain.
- Independent of the computer hardware (portable).
- Translated into machine code using a compiler or an interpreter.
- Widely used for developing applications.

Example:

```
int a = 5;
int b = 10;
int sum = a + b;
```

4. Hierarchy of Translation



5. Examples of High-Level Languages

Language	Type	Main Use
C	Procedural	System programming, education
C++	Multi-paradigm	Systems and applications
Python	Multi-paradigm	Scientific computing, AI, web
Java	Object-oriented	Enterprise and web applications
C#	Object-oriented	Windows and web applications
JavaScript	Scripting	Web development

17. What is C Language?

C is a **high-level programming language** widely used for general-purpose programming, systems development, and education.

1. Definition

C is a general-purpose, procedural programming language developed in the 1970s by Dennis Ritchie at Bell Labs (USA).



2. Main Characteristics

- **High-level language**: easier to write and understand than machine or assembly languages.
- **General-purpose**: used in many domains.
- **Procedural / structured**: programs are organized into functions and procedures.
- **Efficient and fast**: produces efficient executables.
- **Close to the hardware**: allows low-level operations (e.g., memory management).
- **Portable**: programs can run on different operating systems with few modifications.
- **Widely used**: in operating systems, embedded systems, scientific computing, and for teaching programming.

3. Advantages

- Simple and clear syntax.
- Good balance between high-level features and low-level control.
- Efficient and fast execution.
- Rich set of operators and data types.
- Large number of libraries.
- Widely supported and standardised (ANSI/ISO C).

4. C is NOT …

✗	**NOT** an algorithm	C is a language to write programs.
✗	**NOT** a compiler	A compiler translates C programs.
✗	**NOT** a development environment	A development environment (e.g. Code::Blocks) is a tool to write, compile and run C programs.

5. Example of a C Instruction

```c
#include <stdio.h>
int main() {
    printf("Hello, this is a C program!\n");
    return 0;
}
```

This instruction:

- is written in C language.
- displays a message on the screen.
- will be compiled and executed on a computer.

6. Common Uses of C



18. From Algorithm to C Program

A **problem** is solved by an **algorithm**. To make the computer execute this solution, we must write the algorithm in a **programming language** (e.g., C), then **compile** and **run** it.

1. Problem

A well-defined question to be solved.

Example:

Read three numbers and determine the largest one.

2. Algorithm

A finite sequence of steps to solve the problem (independent of any language).

Example:

- Read *a*, *b*, *c*.
- Initialize *max*.
- Compare values.
- Display the maximum.

3. Pseudocode

A structured and language-independent representation of the algorithm.

```
Read a, b, c
max ← a
if  b > max then
    max ← b
end if
if  c > max  then
    max ← c
end if
Print max
```

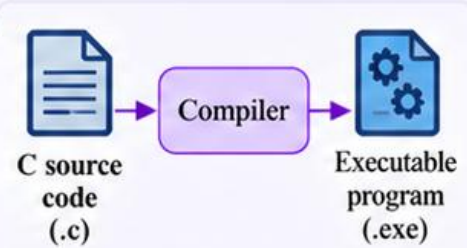
4. C Program

The algorithm is written using the syntax and rules of the C language.

```c
#include <stdio.h>
int main() {
   int a, b, c, max;
   scanf("%d %d %d", &a, &b, &c);
   max = a;
   if (b > max)
      max = b;
   if (c > max)
      max = c;
   printf("Maximum = %d", max);
   return 0;
}
```

5. Compilation and Execution

The source code (C program) is translated by a compiler into machine code (executable program), which is then run on the computer.



6. Overall Process

Problem (what to solve?) → **Algorithm** (solution plan) → **Pseudocode / Flowchart** (representation) → **C Program** (source code) → **Compilation** (compiler) → **Executable program** (machine code) → **Execution** (runs on computer)

19. What is a Compiler?

A **compiler** is a program that translates the source code written in a **high-level programming language** (e.g., C) into **machine code** (an executable program).

1. Definition

- A compiler is a software tool that **translates** an entire program written in a high-level language into machine code.
- It checks the **syntax** of the program and reports **errors**.
- If there are no errors, it generates an **executable file** that can be run on the computer.

2. Role of the Compiler

- Translates the **source code** (e.g., C) into **machine code**.
- Detects and reports **syntax errors**.
- **Optimizes** the code to improve its performance.
- Generates an **executable file** (e.g., .exe).

3. Compilation Process



Source code (C program) main.c → **Compiler** → **Executable program (main.exe)**

4. Example of Compilation

C source code (main.c)

```c
#include <stdio.h>
int main() {
   printf("Hello World!\n");
    return 0;
}
```

→ **Compiler** →

Machine code (executable file)

```
01001110  11001010
11100001  00010111
01010011  11001100
00001010  11110000
...
```

5. Important Notes

- The compiler translates the entire program before execution.
- If there are errors, the compilation stops.
- The generated executable program is machine-dependent (for a specific operating system).

6. Common C Compilers



Code::Blocks (GCC)



Dev-C++ (GCC)



Visual Studio (MSVC)



GCC (Windows, Linux, macOS)

20. What is a Development Environment?

A **development environment** (or **IDE** – Integrated Development Environment) is a software tool that helps us **write, compile, run** and **debug** programs in a programming language (e.g., C).

1. Definition

A development environment is an integrated tool that provides everything needed to develop programs:

- a **text editor** to write the source code,
- a **compiler** to translate the code,
- a way to **run** the executable program,
- and **debugging tools** to detect and fix errors.

2. Main Functions

- Create and edit source files (e.g., .c).
- Compile the program.
- Run the executable program.
- Display error messages.
- Help to debug (find and fix errors).
- Provide useful tools (syntax highlighting, auto-completion, project management, etc.).

3. Common Development Environments for C

Code::Blocks

- Free and open source
- Easy to use
- Popular for C/C++

Dev-C++

- Free
- Simple interface
- Suitable for C beginners

Visual Studio Code

- Modern and flexible
- Needs a compiler (e.g., MinGW)
- Extensions for C/C++

CLion

- Professional IDE
- Advanced features
- Free for students (with license)

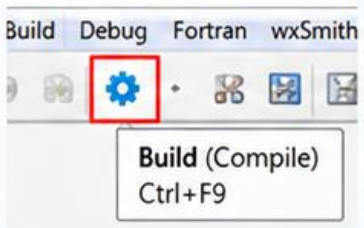
4. Example: Using an IDE (e.g., Code::Blocks)

1. Write the code
Type the C program in the editor.

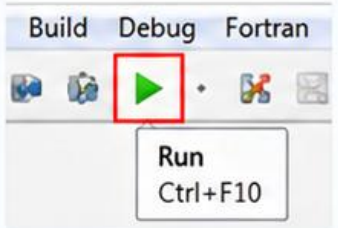
main.c

```
1  #include <stdio.h>
2  int main() {
3      printf("Hello!\n");
4      return 0;
5  }
```

2. Compile
Click on the "Build" button.



3. Run
Click on the "Run" button.



4. View the result
The program executes and displays the output.



5. Important Notes

- A development environment is a tool, not a programming language.
- It usually integrates a text editor, a compiler and a debugger.
- Different IDEs can be used, but the C language remains the same.
- You can also use a simple text editor (e.g., Notepad++) with a separate compiler (e.g., GCC), but an IDE makes the work easier.

21. Putting Everything Together

All the concepts we have seen are connected. The following diagram shows the complete process from a **real problem** to an **executable program**, and the role of each element.

1. Problem



A real situation to be solved. (e.g., find the largest of three numbers).

2. Algorithm



A finite sequence of steps to solve the problem (independent of any language).

3. Pseudocode / Flowchart



A structured representation of the algorithm (language-independent).

4. C Program (Source Code)

```c
#include <stdio.h>
int main() {
   int a, b, c;
   ...
   return 0;
}
```

The algorithm is written in the C programming language using its syntax and rules.

5. Compiler



A program that translates the C source code into machine code and generates an executable file.

6. Executable Program



A machine code program (e.g., .exe) that can be run on a computer.

7. Development Environment (IDE)



An integrated tool that provides a text editor, a compiler, and a way to run and debug C programs (e.g., Code::Blocks, Dev-C++, VS Code).

8. Summary Diagram

Problem (real world question) → **Algorithm** (solution plan) → **Pseudocode / Flowchart** (representation) → **C Program** (source code) → **Compiler** (translation) → **Executable Program** (machine code) → **Run on Computer** (using **IDE**)

9. Key Takeaways

- A **problem** is solved by an **algorithm**.
- An algorithm can be written as **pseudocode** or a **flowchart**.
- A **program** is the implementation of the algorithm in a programming language (here, C).
- A **compiler** translates the program into machine code.
- A **development environment (IDE)** helps us write, compile, run and debug the program.